MailMind

Architecture & User Workflow Reference

Generated from codebase — June 2026

1. High-Level Architecture Overview

MailMind is a two-tier system: a **Next.js 16 / React 19** frontend and a **FastAPI + LangGraph** Python backend. Users authenticate via OAuth (Google or Microsoft), and the backend routes all email operations through a unified mail-provider interface that speaks to either Gmail API or Microsoft Graph API.

Layer 1: Auth & Entry Point

- OAuth Login — Google/Microsoft OAuth 2.0 popup flow, session persistence
- Session State — Active provider (MS/Google), access tokens cached in-process, user principal name
- Mail Provider Client — Microsoft Graph API, Gmail API, unified interface
- Dashboard Entry — Auth check on mount, redirect if unauthenticated, load user profile

Layer 2: Frontend (React / Next.js)

Layout Components

- Sidebar (folder navigation), Header (user profile, theme toggle), Theme toggle

Inbox View

- EmailList, pagination, search/filter, sort options, triage score badges

Email Detail Panel

- ThreadView, DraftPanel (3 styles), triage axis breakdown, PrecedentList

AI Pipeline UI

- Triage score display, CommitmentGate, draft approval, calendar conflict badges, CalendarView

Utilities

- ComposeWindow, TasksView, RAGSettingsView, EvaluationView

API Hooks & State

- <code>useEmails</code> (pagination), <code>useEmailDetail</code> (classification + draft), <code>useCommitments</code>, <code>useCalendar</code>

Layer 3: Backend (FastAPI + LangGraph)

API Routers (6 total)

- Auth Routes — MS/Google OAuth, quick login, logout
- Email Routes — List/get, compose, reply, archive, trash

- AI Routes — Triage (batch), classify, draft generation
- RAG Routes — Retrieve precedents, inject context, RAG settings
- Calendar Routes — Fetch events, create event, commitments
- Compliance Routes — Monitoring/metrics, health/readiness, evaluation

LangGraph Agentic Pipeline (Linear DAG)

```
[START]
   |
   v
ingest_node     <- PII masking, payload validation, queue management
   |
   v
triage_node     <- 5-axis scoring (GPT-4o): deadline, authority, sentiment, decay, action
   |
   v
commitment_node <- Commitment extraction (GPT-4o structured output), deadline detection
   |
   v
calendar_node   <- Calendar conflict detection (deterministic)
   |
   v
rag_node        <- RAG precedent retrieval + context injection + draft generation (GPT-4o)
   |
   v
gate_node       <- Human-in-the-loop approval checkpoint
   |
   v
[END]
```

Core Services

- <code>ClassificationService</code> — GPT-4o email priority (CRITICAL/HIGH/Normal)
- <code>CommitmentService</code> — Structured LLM output for tasks + deadlines
- <code>DraftService</code> — RAG-augmented reply generation
- <code>ToneDNAService</code> — Learn user voice from sent mail
- <code>GraphClient</code> / <code>GmailClient</code> — Unified mail provider interface
- <code>RetrievalService</code> + ChromaDB — Vector similarity search for precedents
- <code>PII Masking</code> — Strip PII before indexing/LLM calls
- <code>Scorers</code> — Deadline, Authority, Sentiment, Decay, ActionType

Infrastructure

- Email Queue (async, webhook + polling)
- Database (optional SQLAlchemy ORM persistence)
- In-process caching (dict-based, TTL-based expiry)
- Middleware: CORS, rate limiting, security headers
- Observability: structured logging, optional Sentry

2. Complete User Workflow

Phase 1: Authentication & Session Initialization

Step 1.1 — User Visits Login Page

```
User navigates to mailmind.com
  v
Frontend loads: app/page.tsx (LoginPage component)
  |-- Checks localStorage for "remembered login"
  |     ■■ rememberMe key + email stored from previous session
  |-- Calls checkAuthStatus() -> GET /api/auth/status
  |     ■■ Backend checks if active session exists
  ■■■ Sets authStatus = "ready"

If authenticated from existing session:
  ■■ Router redirects to /dashboard immediately
```

Step 1.2 — User Selects OAuth Provider

```
User enters email: tarun@gmail.com
  |-- Frontend detects domain -> "gmail" -> provider = "google"
  |-- Frontend opens OAuth popup window (synchronously to preserve gesture)
  ■■■ User clicks "Continue with Google"

Frontend calls: POST /auth/google/login-initiate

Backend (app/services/gmail.py):
  |-- Generates state = random UUID
  |     ■■ Stored in memory: google_auth_status[state]
  |-- Calls build_auth_url(state)
  |     ■■ Returns Google consent URL with:
  |        client_id, redirect_uri, scope, state
  ■■■ Returns: { status: "pending", auth_url: "...", state: "abc123" }

Frontend:
  |-- Navigates popup to auth_url
  ■■■ Starts polling: POST /api/auth/google/poll (every 2.5s)
```

Step 1.3 — OAuth Callback & Token Exchange

```
User completes Google OAuth consent
  ■■ Google redirects to: /api/auth/google/callback?code=AUTH_CODE&state=abc123

Backend (google_callback):
  |-- Calls exchange_code(code)
  |     |-- POST to Google token endpoint
  |     |-- Sends: code, client_id, client_secret, grant_type=authorization_code
  |     ■■ Receives: access_token, refresh_token, expires_in, id_token
  |-- Decodes id_token (JWT) -> extracts email + profile
  |-- Stores refresh token:
  |     ■■ ~/.cache/google_credentials.json  (for Quick Login)
  |-- Updates google_auth_status[state] = { status: "success", email }
  |-- Calls set_provider("google", "tarun@gmail.com")
  ■■■ Returns HTML "Connected" screen -> closes popup after 1.2s

Frontend polling detects success -> router.push("/dashboard")
```

Step 1.4 — Session Persistence

```
Backend storage:
  |-- In-memory: _user_token_cache
  |     ■■ { access_token, refresh_token, expires_at }
  |-- File system: ~/.cache/google_credentials.json
  ■■■ Optional DB: users table { email, provider, token_hash, last_login }

Frontend storage:
  |-- localStorage["remembered_login"] = { email, provider }
  ■■■ sessionStorage: preserved for tab duration
```

Phase 2: Dashboard Load & Inbox Initialization

Step 2.1 — Dashboard Mount

```
Frontend (app/dashboard/page.tsx):
  |-- useEffect on mount -> checkAuthStatus() -> GET /api/auth/status
  |     ■■ Backend validates token expiry, returns user_principal_name
  |-- If authenticated:
  |     |-- setAuthenticated(true), setUserEmail, setProvider
  |     |-- userStorage.setUser("tarun@gmail.com")
  |     |     ■■ Scopes all localStorage to this user
  |     |         (prevents cross-account data leaks)
  |     ■■ setCheckingAuth(false) -> stops loading spinner
  ■■■ If not authenticated -> router.push("/")
```

Step 2.2 — Initial Email List Fetch

```
useEmails hook -> fetchEmails()
  ■■■ POST /api/mailbox?folder=inbox&limit=50

Backend (get_mailbox):
  |-- Validates user via Depends(get_current_user)
  |-- Calls client = get_mail_client() -> singleton GmailClient
  |-- Calls client.list_emails(folder="inbox", limit=50)
  |     ■■ GET https://www.googleapis.com/gmail/v1/users/me/messages
  |            ?q=in:inbox&maxResults=50
  ■■■ Returns EmailPage: { emails: [50 items], page_token: "...", has_next: true }
```

Step 2.3 — Triage Score Batch Fetch

```
Frontend sends POST /api/triage/batch with 50 email payloads

Backend (triage_batch):
  |-- Creates shared scorers dict (reused across batch)
  |-- For each email -> cache check:
  |     Key: f"id:{current_user}:{email_id}"
  |     If HIT  -> return cached TriageResult instantly
  |     If MISS -> compute all 5 axes:
  |
  |  1. DeadlineScorer
  |     Regex: "by 3pm", "EOD", "ASAP"
  |     < 4 hours  -> score 1.0 (CRITICAL)
  |     < 24 hours -> score 0.7
  |
  |  2. SenderAuthorityScorer
  |     CEO/manager domain  -> 0.9
  |     Unknown sender      -> 0.2
  |
  |  3. SentimentScorer  (spaCy NLP)
  |     "urgent","ASAP"     -> 0.9
  |     Positive tone       -> 0.3
  |
  |  4. ThreadAgeDecayScorer
  |     < 1 hour old        -> 1.0
  |     > 24 hours old      -> 0.3
  |
  |  5. ActionTypeScorer
  |     "FYI"               -> 0.2
  |     "Review"            -> 0.6
  |     "Approval needed"   -> 0.9
  |
  |-- CompositeAggregator: weighted average of 5 axes
  |     >= 0.75  -> "Critical"  (red badge)
  |     0.5-0.75 -> "High"      (orange badge)
  |     < 0.5    -> "Normal"    (gray badge)
  |
  ■■■ Cache write: triage_cache.set(key, result)  [TTL: 1 hour]
      Optional DB write: triage_cache table
```

Phase 3: User Opens Email — Full AI Pipeline

Step 3.1 — Email Selection & Mark as Read

```
User clicks email -> handleSelectEmail("18a...")
  |-- setSelectedEmailId("18a...")
  |-- If email.isRead === false -> POST /api/emails/18a.../read
  |     Backend: GmailClient.mark_read()
  |       -> POST Gmail API { removeLabelIds: ["UNREAD"] }
  |       -> Optional DB: emails table UPDATE is_read = true
  ■■■ EmailDetail mounts with selectedEmail data
```

Step 3.2 — 6 Parallel AI Pipeline Calls (Promise.all)

All 6 calls fire simultaneously via Promise.all in the useEmailDetail hook. Results are merged into the UI when all resolve (~1.5s).

Call 1 — Classification

```
POST /api/classify { email_text: "[full email]" }

Backend (ClassificationService.classify):
  |-- Cache check: f"classify:{user}:sha256(email_text)"
  |-- If MISS: GPT-4o call -> "Classify as CRITICAL, HIGH, or NORMAL"
  |-- Cache write: classification_cache.set(key, result) [TTL: 1 hour]
  ■■■ Optional DB write: classifications table
```

Call 2 — Triage

```
POST /api/triage { email_payload }

Backend: Returns TriageResult
  ■■ Usually already cached from the batch fetch -> returns instantly
```

Call 3 — Commitment Extraction

```
POST /api/commitments/extract { email_text, thread_summary, email_id }

Backend (CommitmentService.extract):
  |-- PII masking (replace emails, SSN, credit cards with [PII_*])
  |-- GPT-4o structured output:
  |     Returns JSON:
  |     { commitments: [
  |         { id, text: "Call client by 3pm",
  |           deadline, type: "call", confidence: 0.95 },
  |         { id, text: "Submit report",
  |           deadline, type: "deliverable", confidence: 0.87 }
  |     ]}
  ■■■ Optional DB write: commitments table (status: "pending")
```

Call 4 — Calendar Fetch

```
GET /api/calendar?days=7

Backend (CalendarFetcher):
  |-- GmailClient.fetch_calendar()
  |     -> GET https://www.googleapis.com/calendar/v3/calendars/primary/events
  ■■■ Caches result for 5 minutes: calendar_cache[user_id]
```

Call 5 — RAG Precedent Retrieval

```
POST /api/rag/retrieve { email_text: "[masked email]" }

Backend (RetrievalService):
  |-- Cache check: f"retrieve:{user}:sha256(email_text)"
  |-- If MISS:
  |     |-- Convert email_text to 1536-dim embedding (OpenAI)
  |     |-- Query ChromaDB stored at ./chroma_db/
  |     |     chroma_db/
  |     |       |-- index.json
  |     |       |-- chroma.parquet
  |     |       ■■■ data/documents.json
  |     |-- Cosine similarity search -> top-5 above threshold
  |     ■■■ Returns PrecedentItem[] with similarity scores
  |-- Cache write: precedents_cache.set(key, results)
  ■■■ Optional DB write: retrieval_logs table
```

Call 6 — Draft Generation

```
POST /api/rag/draft { email_text, style: "standard", sender, subject,
                      current_user_email }

Backend (DraftService.generate_draft):
  |-- Cache check: f"draft:{style}:{user}:sha256(email_text)"
  |-- If MISS:
  |     |-- Load ChromaDB index from disk
  |     |-- PrecedentInjector.inject():
  |     |     Includes original email + 2 similar past emails + user replies
  |     |-- Load Tone DNA profile:
  |     |     ~/.mailmind/tone_dna/tarun@gmail.com.json
  |     |     { formality_score: 0.72, avg_sentence_length: 16,
  |     |       favorite_phrases: ["looking forward", "let me know"],
  |     |       tone_markers: { friendliness: 0.8, urgency: 0.3 } }
  |     |-- GPT-4o call (temperature 0.3)
  |     ■■■ Generates 3 style variants:
  |           Standard:  "Thanks for the update. Ready by 3pm."
  |           Formal:    "Thank you. I will ensure completion by 15:00."
  |           In-Depth:  "Thank you for the update. I appreciate..."
  |-- Cache write: precedents_cache.set(key, DraftResponse)
  ■■■ Optional DB write: draft_cache table
```

Step 3.3 — Tone DNA Build (First Use)

```
POST /api/tone-dna/build

Backend (ToneDNAService.ingest_and_build):
  |-- Fetch last 90 days of sent emails
  |-- Analyze 50+ emails for:
  |     |-- Formality score (formal vs casual word ratio)
  |     |-- Avg sentence length
  |     |-- Favorite phrases (n-grams appearing 3+ times)
  |     ■■■ Tone markers via spaCy NLP
  |-- Generate profile JSON and write to disk:
  |     ~/.mailmind/tone_dna/tarun@gmail.com.json
  |     { formality_score: 0.72, sample_size: 67,
  |       favorite_phrases: ["looking forward to", "let me know"],
  |       tone_markers: { urgency: 0.3, friendliness: 0.8 } }
  ■■■ Optional DB write: tone_dna_profiles table
```

Step 3.4 — Calendar Conflict Detection

```
Frontend (CommitmentGate):
  |-- Commitment: "Call client by 3pm" -> deadline 2024-06-11T15:00:00Z
  |-- Calendar event: "Client call" 3pm-4pm
  ■■■ Detects overlap -> ConflictBadge: "Conflicts with 'Client call' at 3pm"

Backend (optional): check_calendar_conflict() in calendar_node
  ■■ Marks commitment as has_conflict = true
```

Phase 4: User Approves Commitments

Step 4.1 — CommitmentGate UI

```
Frontend displays:
  |-- [x] "Call client by 3pm"  [!] (conflicts with existing meeting)
  ■■■ [x] "Submit report"       (Tomorrow 9am)

User reviews, unchecks items to skip, clicks "Confirm"
  ■■ POST /api/commitments/confirm  + X-Approval-Token header
```

Step 4.2 — Create Calendar Events & Tasks

```
Backend (CommitmentService.confirm):
  |-- Validates X-Approval-Token header (CSRF protection)
  |-- For each approved commitment:
  |     If type == "call" / "meeting":
  |       -> Creates calendar event via Gmail/Graph API
  |          POST https://www.googleapis.com/.../events
  |          { title, start, end, description, source: "MailMind" }
  |     If type == "deliverable" / "task":
  |       -> Creates task in Google Tasks / Microsoft To Do
  |          POST .../tasks  { title, dueDateTime, body }
  |-- DB writes:
  |     commitments table UPDATE:
  |       status -> "confirmed", calendar_event_id, task_id,
  |       confirmed_at, confirmed_by
  ■■■ audit_logs table:
      { action: "commitment_confirmed", user_id, email_id, timestamp }

Frontend: Toast "2 items added to calendar" + task/event links
```

Phase 5: Draft Approval & Send

Step 5.1 — User Reviews Draft

```
DraftPanel shows 3 style tabs:
  Standard:  "Thanks for the update. I'll have everything ready by 3pm."
  Formal:    "Thank you for the notification. Completion by 15:00."
  In-Depth:  "Thank you for the update. I appreciate the clarity..."

User can: switch styles, edit text inline, view precedent citations, send
```

Step 5.2 — Send Reply

```
POST /api/emails/18a.../reply { comment: "[edited draft]" }

Backend:
  |-- GmailClient.send_reply()
  |     |-- Constructs reply with In-Reply-To + References headers
  |     ■■■ POST https://www.googleapis.com/gmail/v1/users/me/messages/send
  |-- DB writes:
  |     sent_emails:  { message_id, user_id, to, subject, body, replied_to, sent_at }
  |     emails:       UPDATE { replied: true, reply_sent_at }
  ■■■ Analytics: draft_analytics table
      { email_id, style_selected, time_to_send_ms }

Frontend: Toast "Reply sent" -> inbox refreshes
```

Phase 6: Pagination

```
User scrolls to bottom of email list
  ■■ useEmails hook detects near-end
  ■■ nextPage() -> fetchEmails(page_token="abc123def456")

Backend: Gmail API resumes from token -> returns emails 51-100

Frontend:
  |-- Appends 50 more emails to array
  ■■■ POST /api/triage/batch for new emails (cache hits where possible)
```

Phase 7: Session Persistence & Auto-Resume

```
User returns next day (in-process token cache cleared by server restart):

Frontend: localStorage["remembered_login"] found -> calls checkAuthStatus()

Backend:
  |-- _user_token_cache is empty
  |-- Reads ~/.cache/google_credentials.json -> extracts refresh_token
  |-- POST https://oauth2.googleapis.com/token
  |     grant_type=refresh_token -> receives new access_token
  |-- Updates _user_token_cache with new token
  ■■■ Returns { authenticated: true, user_principal_name: "tarun@gmail.com" }

Frontend: quickLogin() -> router.push("/dashboard")  (no re-auth screen)
  ■■ userStorage.setUser() restores scoped preferences from previous session
```

3. Caching Strategy Summary

Layer 1: Browser localStorage (Frontend)

Key	Value	Persistence
`remembered_login`	{ email, provider }	Until cleared
`userStorage:{user}`	Search filters, sort, theme	Until cleared

Layer 2: In-Process Memory (Backend)

Cache	Key Pattern	TTL
triage_cache	id:{user}:{email_id}	1 hour
classification_cache	classify:{user}:{sha256}	1 hour
precedents_cache	retrieve:{user}:{sha256}	Indefinite
calendar_cache	{user_id}	5 minutes
google_auth_status	{state}	Until OAuth completes
ms_auth_status	{state}	Until OAuth completes

Layer 3: File System (Backend)

Path	Content
~/.cache/google_credentials.json	Google refresh token
~/.mailmind/tone_dna/{user}.json	Tone DNA profile

Layer 4: Database (Optional — if DATABASE_URL set)

Tables: emails, triage_cache, classifications, commitments, tone_dna_profiles, audit_logs, draft_cache, retrieval_logs

Cache Invalidation Rules

- RAG index — Manual refresh via POST /api/rag/settings
- Tone DNA — Manual rebuild via POST /api/tone-dna/build
- Triage / Classification — Auto-expire after 1 hour
- Calendar — Auto-expire after 5 minutes

4. End-to-End Timeline (Single Email)

```
T+0ms:      User clicks email
T+50ms:     Mark as read -> POST /api/emails/{id}/read
T+150ms:    6 parallel AI calls start:
              POST /api/classify             (~500ms)
              POST /api/triage               (~100ms, cached)
              POST /api/commitments/extract  (~1200ms, LLM)
              GET  /api/calendar             (~300ms)
              POST /api/rag/retrieve         (~800ms, embedding + ChromaDB)
              POST /api/rag/draft            (~1500ms, LLM)
T+1500ms:   All calls complete -> full EmailDetail renders
            Triage badge + axes, commitments, calendar conflicts,
            draft (3 styles), precedent citations
T+5000ms:   User clicks "Confirm Commitments"
              -> Calendar events + tasks created (~700ms)
T+5700ms:   User clicks "Send" on draft
              -> Reply delivered via Gmail API (~200ms)
T+5900ms:   Toast: "Reply sent, 2 items added to calendar"

Total: ~6 seconds for the full workflow (LLM + mail provider calls)
```

5. External Integrations

Integration	Purpose
Microsoft Graph API	Outlook mail, Calendar, Teams, To Do
Gmail API	Gmail mail, Google Calendar, Google Tasks
Azure OpenAI (GPT-4o)	Classification, triage, commitment extraction, draft generation
ChromaDB	Local/cloud vector DB for RAG precedent retrieval
spaCy (en_core_web_sm)	NLP for sentiment scoring and tone analysis
Vercel Analytics	Frontend observability
Sentry (optional)	Backend error tracking and performance monitoring